From Word Embedding to Contextual Embedding

RNN, LSTM, and GRU

Professor Anis Koubaa

MAI554: Deep Learning for NLP
Alfaisal University
akoubaa@alfaisal.edu

Spring 2025



Outline

1. Motivation: Why Contextual Embeddings?
2. Text Pre-Processing for Sequences
3. Recurrent Neural Networks (RNN)
4. Hands-On Traces: Breaking the Abstraction
5. Training RNNs and Their Limitations
6. Long Short-Term Memory (LSTM)
7. Gated Recurrent Unit (GRU)
8. Comparison and Outlook

Recap: Static Embeddings

What We Have So Far

Static models assign **one fixed vector per word**, regardless of context.

- **Word2Vec** – predict neighbours (Skip-gram / CBOW)
- **GloVe** – global co-occurrence statistics
- **FastText** – subword n-grams for rare words

The Problem

"The **bank** loaned me money." vs. "The river **bank** was muddy."

Same word ⇒ same vector!



The Need for Contextual Embeddings

Key Insight

The meaning of a word depends on the words around it. We need embeddings that **change with context**.

Static Embedding:

$$\text{Embed}(\text{“bank”}) = \boldsymbol{v}_{\text{bank}} \quad \text{(always the same)}$$

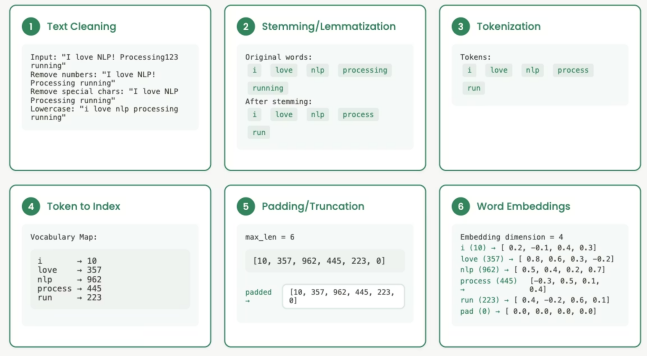
Contextual Embedding:

$$\boldsymbol{h}_t = f(\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_t)$$

The vector for “bank” now *depends on the sentence*.

Solution: Process the sequence with a recurrent model that accumulates context.

Text Pre-Processing Pipeline



1–3: Clean & Tokenise
Remove noise, stem, split.

4–5: Encode & Pad
Map to indices; pad to uniform length.

6: Embed
Look up dense vectors.

Step-by-Step: Clean and Tokenise

Raw Input

```
"I LOVED this movie!!!  #awesome :)"
```

Step 1 — Clean

Remove punctuation, special characters, lower-case.

```
"i loved this movie awesome"
```

Step 2 — Tokenise

Split the string into a list of tokens.

```
["i", "loved", "this", "movie", "awesome"]
```

Why Clean?

- Noise (HTML tags, emojis, punctuation) adds useless vocabulary
- Reduces vocab size ⇒ faster training
- "Movie", "MOVIE", "movie" should be the same token

Why Tokenise?

- Neural networks cannot read raw strings
- Need discrete units to map to numbers
- Choice matters: word-level, subword (BPE), character-level

Step-by-Step: Encode, Pad, and Embed

Step 3 — Encode

Map each token to an integer index via a vocabulary.

```
vocab = {i:1, loved:2, this:3, movie:4, awesome:5}
tokens → [1, 2, 3, 4, 5]
```

Step 4 — Pad

Make all sequences the **same length** for batching.

```
Seq A: [1, 2, 3, 4, 5]  (len 5)
Seq B: [6, 7]  (len 2)
After pad:  [6, 7, 0, 0, 0]
```

Step 5 — Embed

Look up each index in an **embedding matrix** $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d}$.

index 2 → $\mathbf{E}[2]$ = [0.31, −0.12, ...] $\in \mathbb{R}^d$

Why Encode?

- Math operates on numbers, not strings
- Index = row address in embedding table

Why Pad?

- GPU batches must be rectangular tensors
- Masking prevents <PAD> affecting gradients

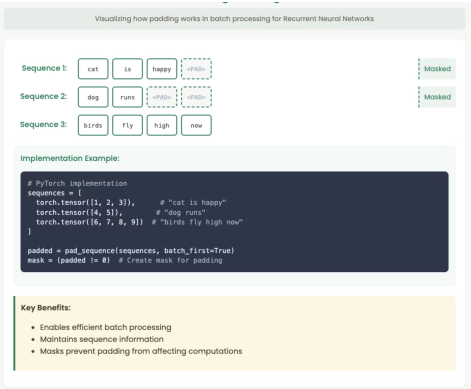
Why Embed?

- Plain integers carry no semantic meaning
- Dense vectors capture similarity: "loved" ≈ "enjoyed"
- Embeddings are **learnable** — improve during training

Padding and Masking

Why Padding?

- RNNs process **batches** of sequences
- Pad shorter ones with <PAD> token



Why Masking?

- Prevent <PAD> tokens from affecting learning
- Binary mask: $m_t = \mathbb{1}[x_t \neq 0]$

PyTorch Pattern

```
padded = pad_sequence(seqs,
batch_first=True)
mask = (padded != 0)
```

Intuition: Why Do We Need Memory in Neural Networks?

The Core Problem

A feed-forward NN sees each input **in isolation** — no memory of what came before.

Analogy: Reading a book but **forgetting** after each word.

- "The cat sat on the _____" — need "cat" to predict "mat"
- "I grew up in France... I speak fluent _____" — need "France"!

The Idea Behind RNNs

Pass a "hidden state" $\boldsymbol{h}_t$ that carries a summary of everything seen so far.

Feed-Forward
"No memory"

Sees: "mat"
Knows: nothing else

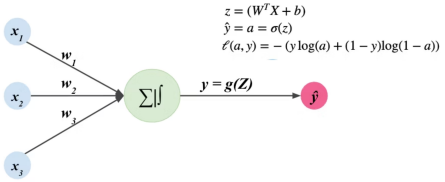
Recurrent
"Has a notebook"

Sees: "mat"
Remembers: "The cat sat on the..."

Step 1: The Perceptron — A Single Neuron

Simplest Neural Network

One neuron: weighted sum → activation → output.



$$z = \mathbf{W}^\top \boldsymbol{x} + b$$
$$\hat{y} = \sigma(z)$$

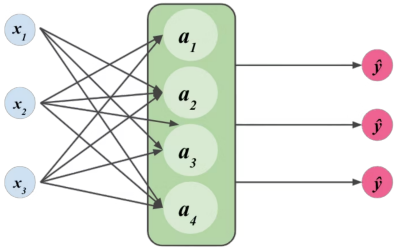
- Inputs x_1, x_2, x_3 with weights w_1, w_2, w_3
- Single output $\hat{y}$ (e.g. binary classification)
- No hidden layer, no memory

Step 2: Feed-Forward Neural Network

Adding a Hidden Layer

Stack many neurons into layers. Each layer transforms the input.

$$\hat{\boldsymbol{y}} = g\big(\mathbf{W}_2 \cdot \sigma(\mathbf{W}_1 \boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2\big)$$

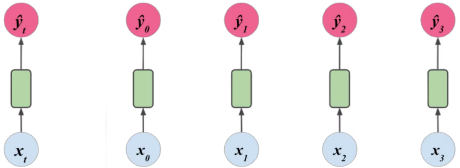


- Inputs $\rightarrow$ hidden layer $\rightarrow$ outputs
- Can learn **non-linear** patterns
- But each input is still processed **independently** — no notion of sequence or time

Step 3: What About Sequences?

FFN on Temporal Data

If we apply the same network to each time step independently:



$$\hat{y}_t = f(x_t) \quad \text{(no connection between steps)}$$

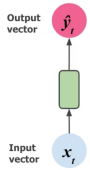
- Each x_t is processed in **isolation**
- The network at t=3 has **no idea** what happened at t=0, 1, 2
- Useless for language: "The cat sat on the _____" requires remembering "cat"!

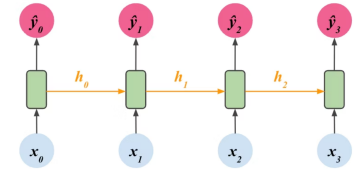
We need connections between time steps...

Step 4: Adding Memory ⇒ The Recurrent Neural Network

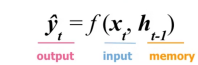
The Key Idea

Pass a **hidden state** $\boldsymbol{h}_t$ from each step to the next. Now the output depends on **input** + **history**.



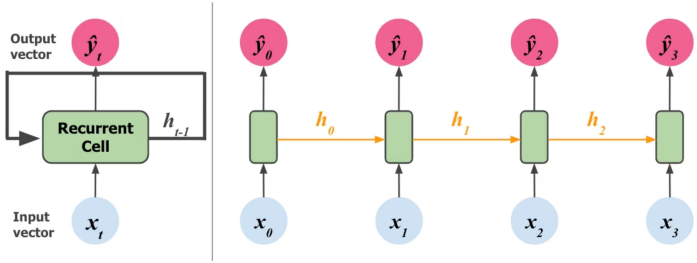


$$\hat{\boldsymbol{y}}_t = f(\boldsymbol{x}_t, \boldsymbol{h}_{t-1})$$



- $\boldsymbol{h}_t$ is a **summary** of all past inputs
- Same weights shared across all time steps
- Now "sat" knows about "cat"!

RNN: Folded vs Unfolded View



Folded (compact)

One box with a self-loop. The arrow from h_{t-1} back into the cell shows **recurrence**.

Unfolded (through time)

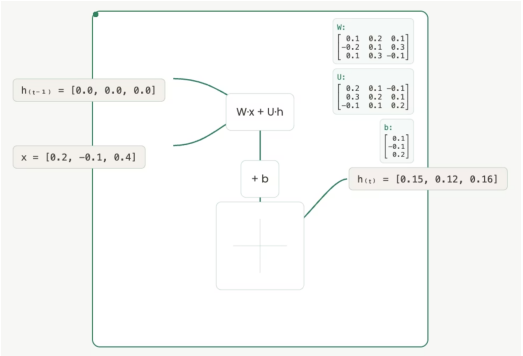
Same cell copied at each time step. Hidden states $h_0 \rightarrow h_1 \rightarrow h_2$ flow left to right.

RNN: The Core Equation

Hidden State Update

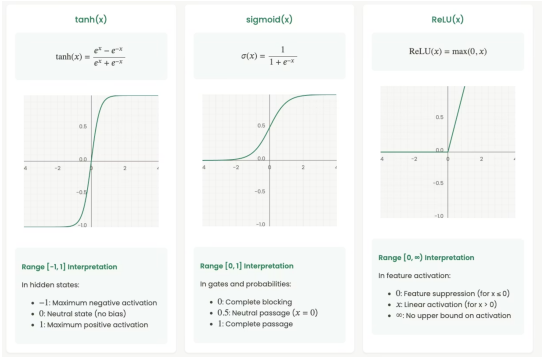
$$\boldsymbol{h}_t = \tanh\big(\mathbf{W}_x \boldsymbol{x}_t + \mathbf{W}_h \boldsymbol{h}_{t-1} + \boldsymbol{b}\big) \qquad (1)$$

$\boldsymbol{x}_t \in \mathbb{R}^m$	current input (word embedding)
$\boldsymbol{h}_{t-1} \in \mathbb{R}^d$	previous hidden state
$\mathbf{W}_x \in \mathbb{R}^{d \times m}$	input-to-hidden weights
$\mathbf{W}_h \in \mathbb{R}^{d \times d}$	hidden-to-hidden weights
$\boldsymbol{b} \in \mathbb{R}^d$	bias



- **Same parameters** $(\mathbf{W}_x, \mathbf{W}_h, \boldsymbol{b})$ are shared across all time steps
- tanh keeps values in $[-1, 1]$

Activation Functions in RNNs



tanh – Range $[-1, 1]$
Zero-centred. RNN hidden states.

Sigmoid – Range $[0, 1]$
Probabilities. LSTM/GRU gates.

ReLU – Range $[0, \infty)$
Fast. Rarely used in RNNs.

Why This Activation? Design Rationale

tanh → RNN Hidden States

Range $[-1, +1]$, **zero-centred**.

Why?

- Positive *and* negative outputs
- -0.8 vs $+0.9$ encodes opposing signals (e.g. neg. vs. pos. sentiment)
- Zero-centred ⇒ no systematic gradient bias ⇒ stable training

Use: RNN / LSTM / GRU hidden states

Sigmoid → Gates & Vision

Range $[0, 1]$, a **probability / switch**.

Why?

- 0 = **block**; 1 = **pass through**
- LSTM/GRU gates: forget, input, output
- Vision: suppress uninformative pixels (0), keep relevant ones (1)
- Binary classification output

Use: gates, classifiers, pixel masks

ReLU → Deep & Conv Nets

Range $[0, \infty)$, **sparse & fast**.

Why?

- Negative → **exact zero**: suppresses irrelevant features
- No saturation ⇒ no vanishing gradient in 100+ layer stacks
- Cheap: just $\max(0, x)$
- **Caution in RNNs:** unbounded ⇒ explodes across time steps

Use: CNNs, feed-forward, deep nets

Takeaway: Activation = *design choice*. Positive/negative → **tanh**. On/off gates → **sigmoid**. Sparse deep features → **ReLU**.

Deep RNNs: Stacking Layers

Why Stack?

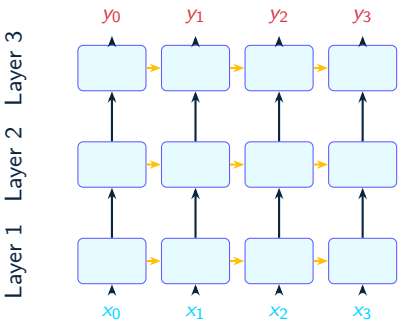
A single-layer RNN learns *one level* of abstraction. Stacking L layers lets the network learn **hierarchical features** — just like deep CNNs.

How it works:

- Layer 1 reads word embeddings $\boldsymbol{x}_t$
- Layer ℓ reads the hidden states of layer $\ell-1$
- Each layer has its **own weights**

$$\boldsymbol{h}_t^{(1)} = \tanh\big(\mathbf{W}_x^{(1)}\boldsymbol{x}_t + \mathbf{W}_h^{(1)}\boldsymbol{h}_{t-1}^{(1)} + \boldsymbol{b}^{(1)}\big)$$
$$\boldsymbol{h}_t^{(\ell)} = \tanh\big(\mathbf{W}_x^{(\ell)}\boldsymbol{h}_t^{(\ell-1)} + \mathbf{W}_h^{(\ell)}\boldsymbol{h}_{t-1}^{(\ell)} + \boldsymbol{b}^{(\ell)}\big)$$

Typical depth: 2–4 layers. ELMo uses 2 BiLSTM layers.



Orange = hidden state flow (time).

Blue = input/output flow (depth).

Bidirectional RNNs: Looking Both Ways

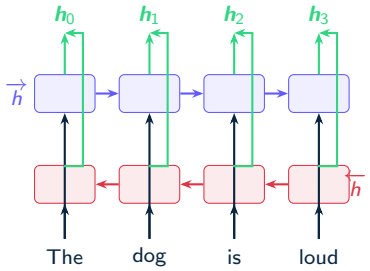
The Problem with Unidirectional RNNs

$\boldsymbol{h}_t$ only sees words $x_1, \ldots, x_t$ (the **past**). But meaning often depends on **future** words too!

Example:
"The _____ is barking loudly" — to understand position 2, you need "barking" (position 4).

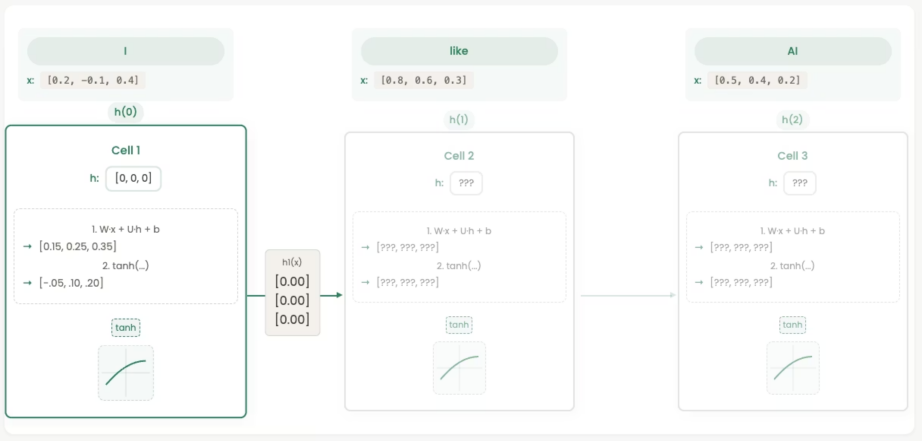
Solution: Two RNNs in Parallel

$$\overrightarrow{\boldsymbol{h}}_t = \text{RNN}_{\text{fwd}}(\boldsymbol{x}_t, \overrightarrow{\boldsymbol{h}}_{t-1})$$
$$\overleftarrow{\boldsymbol{h}}_t = \text{RNN}_{\text{bwd}}(\boldsymbol{x}_t, \overleftarrow{\boldsymbol{h}}_{t+1})$$
$$\boldsymbol{h}_t = [\overrightarrow{\boldsymbol{h}}_t ; \overleftarrow{\boldsymbol{h}}_t] \text{ (concat, dim } 2d)$$



Each $\boldsymbol{h}_t$ sees the **entire sentence**, not just the past.

RNN Trace: Sentiment Analysis Setup



Setup

Input: "**I like AI**" $\Rightarrow$ $\boldsymbol{x}_1 = [0.2, -0.1, 0.4]$, $\boldsymbol{x}_2 = [0.8, 0.6, 0.3]$, $\boldsymbol{x}_3 = [0.5, 0.4, 0.2]$

RNN Trace: Forward Pass

Step 1: "I"

$\boldsymbol{h}_0 = [0, 0, 0]$

$\boldsymbol{z}_1 = \mathbf{W}_x \boldsymbol{x}_1 + \mathbf{W}_h \boldsymbol{h}_0 + \boldsymbol{b}$
$= [0.15, 0.25, 0.35]$

$\boldsymbol{h}_1 = \tanh(\boldsymbol{z}_1)$
$= [-0.05, 0.10, 0.20]$

Step 2: "like"

$\boldsymbol{z}_2 = \mathbf{W}_x \boldsymbol{x}_2 + \mathbf{W}_h \boldsymbol{h}_1 + \boldsymbol{b}$
$= [0.35, 0.25, -0.15]$

$\boldsymbol{h}_2 = \tanh(\boldsymbol{z}_2)$
$= [0.30, 0.20, -0.10]$

Step 3: "AI"

$\boldsymbol{z}_3 = \mathbf{W}_x \boldsymbol{x}_3 + \mathbf{W}_h \boldsymbol{h}_2 + \boldsymbol{b}$
$= [0.45, 0.15, 0.30]$

$\boldsymbol{h}_3 = \tanh(\boldsymbol{z}_3)$
$= [0.40, 0.10, 0.25]$

$\boldsymbol{h}_3$ encodes the **entire sentence** — use it for classification.

RNNs Produce Contextual Embeddings

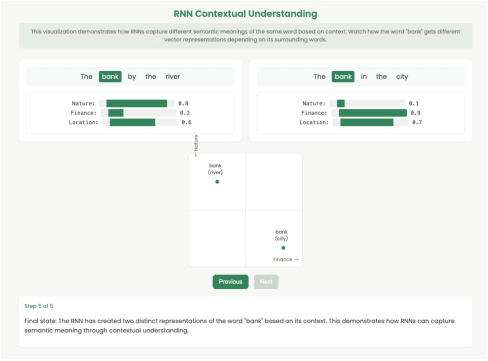
How It Works

Each hidden state $\boldsymbol{h}_t$ is a **contextual embedding** — it encodes word t *plus all preceding words*.

Example: the word "bank"

- "The **bank** by the river" $\Rightarrow \boldsymbol{h}_{\text{bank}}$ emphasises *nature*
- "The **bank** in the city" $\Rightarrow \boldsymbol{h}_{\text{bank}}$ emphasises *finance*

Same word, **different vectors** — because $\boldsymbol{h}_t$ depends on $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_t$.



Applications of Recurrent Models

Sequence → Label

Use final $\boldsymbol{h}_T$ to classify the whole sequence.

- **Sentiment analysis**: "I love this movie" → positive
- **Spam detection**: email → spam/ham
- **Document classification**

Sequence → Sequence (same length)

Use every $\boldsymbol{h}_t$ to label each token.

- **Named Entity Recognition**: "*Barack*/PER *Obama*/PER *visited*/O *Paris*/LOC"
- **POS tagging**: noun, verb, adj...

Sequence → Sequence (different length)

Encoder–Decoder (Seq2Seq) architecture.

- **Machine translation**: "Je suis étudiant" → "I am a student"
- **Summarisation**: long text → summary
- **Chatbots**: question → answer

Language Modelling

Predict next token: $P(x_{t+1} \mid x_1, \ldots, x_t)$.

- **Text generation**
- **Speech recognition**
- Pre-training for downstream tasks

Encoder-Decoder (Seq2Seq) Architecture

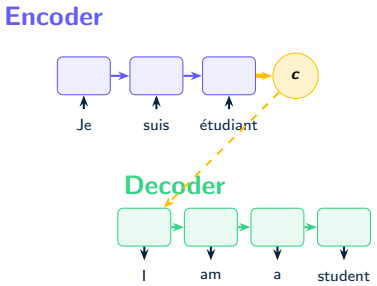
The Idea

For tasks where input and output have **different lengths** (e.g. translation), split the model into two parts.

1. **Encoder** reads the input sequence and compresses it into a fixed-size **context vector** $\boldsymbol{c} = \boldsymbol{h}_T^{\text{enc}}$
2. **Decoder** generates the output sequence one token at a time, conditioned on $\boldsymbol{c}$

The Bottleneck Problem

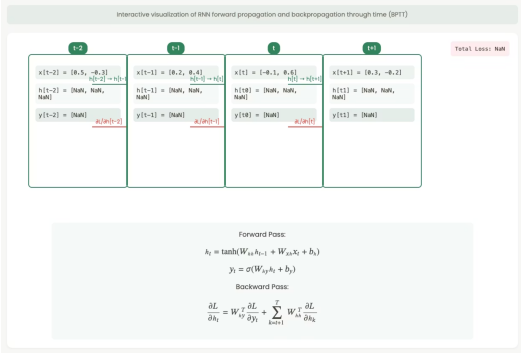
The entire input (no matter how long) must fit into **one vector** $\boldsymbol{c}$. For a 50-word sentence, that is a lot of compression! This motivates **attention** (next section).



Backpropagation Through Time (BPTT)

Training an RNN

Unroll the network across T time steps and apply standard backpropagation.



Gradient of the loss w.r.t. $\mathbf{W}_h$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_h} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial \mathbf{W}_h}$$

Each term involves a **chain of multiplications**:

$$\frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{h}_k} = \prod_{i=k+1}^{t} \frac{\partial \boldsymbol{h}_i}{\partial \boldsymbol{h}_{i-1}} = \prod_{i=k+1}^{t} \mathbf{W}_h^\top \operatorname{diag}\big(\tanh'(\boldsymbol{z}_i)\big)$$

BPTT Intuition: Why Does the Gradient Shrink or Explode?

The Core Idea (No Math!)

Training = "how much should I **blame** each past input for today's error?" To send blame back in time, we **multiply a number at every step**.

Analogy: the telephone game

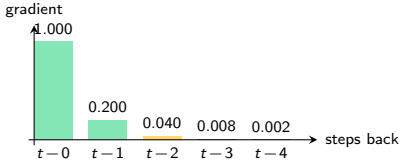
Each person **multiplies** what they hear by a factor f:

- $f = 0.5$: $1 \to 0.5 \to 0.25 \to 0.125 \to \not\approx 0$ Vanishes!
- $f = 2.0$: $1 \to 2 \to 4 \to 8 \to \infty$ Explodes!
- $f = 1.0$: $1 \to 1 \to 1 \to 1$ Preserved (hard!)

Numerical Example

tanh derivative ≈ 0.4, weight factor ≈ 0.5
Per-step multiplier: $0.5 \times 0.4 = \mathbf{0.2}$

Steps back	Gradient scale	
1	$0.2^1 = 0.200$	Useful
2	$0.2^2 = 0.040$	Small
5	$0.2^5 = 0.00032$	≈ 0
10	$0.2^{10} \approx 10^{-7}$	Gone



Vanilla RNNs cannot learn long-range dependencies.
LSTM/GRU gates keep the multiplier ≈ 1.

The Vanishing & Exploding Gradient Problem

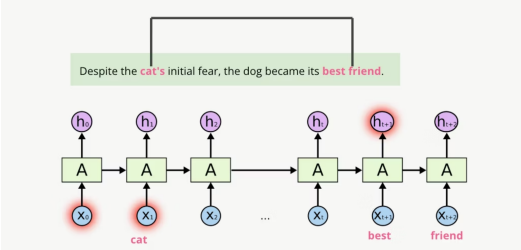
The Product of Jacobians

$$\prod_{i=k+1}^{t} \mathbf{W}_h^\top \operatorname{diag}\big(\tanh'(\boldsymbol{z}_i)\big)$$

If $\|\mathbf{W}_h\| < 1$: gradients **vanish** exponentially
If $\|\mathbf{W}_h\| > 1$: gradients **explode** exponentially

Consequences:

- Cannot learn **long-range dependencies**
- Early words have negligible influence on later outputs
- Gradient clipping helps explosions, but *not* vanishing



"Despite the **cat***'s initial fear, the dog became its best* **friend***."*

The gradient from "friend" barely reaches "cat".

Gradient Clipping: A Practical Fix for Exploding Gradients

The Idea

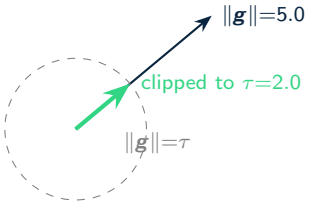
If the gradient norm exceeds a threshold τ, **rescale** it so its norm equals τ. Direction is preserved, magnitude is capped.

$$\boldsymbol{g} \leftarrow \begin{cases} \boldsymbol{g} & \text{if } \|\boldsymbol{g}\| \leq \tau \\ \dfrac{\tau}{\|\boldsymbol{g}\|} \cdot \boldsymbol{g} & \text{if } \|\boldsymbol{g}\| > \tau \end{cases}$$

PyTorch (2 lines)

```
loss.backward()
# Clip before optimizer step
torch.nn.utils.clip_grad_norm_(
  model.parameters(),
  max_norm=1.0
)
optimizer.step()
```

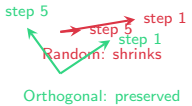
- Fixes exploding gradients — training no longer diverges
- Does NOT fix vanishing gradients — a small gradient stays small
- Typical τ: 1.0 to 5.0



Background: What Do These Initialisation Terms Mean?

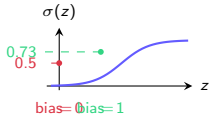
Orthogonal Init (for $\mathbf{W}_h$)

Rows are perpendicular ⇒ multiplying a vector **preserves its length**. Crucial for $\mathbf{W}_h$ which is multiplied at *every* time step.



Forget Gate Bias = 1.0 (LSTM)

Forget gate: "how much old memory to keep?"
bias = 0: $\sigma(0) = 0.5$ ⇒ erases half before learning!
bias = 1: $\sigma(1) \approx 0.73$ ⇒ keeps most, learns what to forget.



Xavier / Glorot Init (for $\mathbf{W}_x$)

Random weights scaled by $\frac{1}{\sqrt{\text{fan_in}+\text{fan_out}}}$. Keeps activations and gradients in a reasonable range. Named after Xavier Glorot (2010).

Quick Rule of Thumb

- $\mathbf{W}_h \rightarrow$ **Orthogonal** (preserve gradient norm)
- $\mathbf{W}_x \rightarrow$ **Xavier** (balance layer scales)
- LSTM forget bias → **1.0** (start remembering)

Practical Training Tips for RNNs

Weight Initialisation

- $\mathbf{W}_h$: use **orthogonal** init (preserves gradient norm)
- $\mathbf{W}_x$: use **Xavier/Glorot** init
- Biases: zero, except forget gate bias $\rightarrow$ **1.0** (start by remembering everything)

Dropout in RNNs

- Never apply dropout *between time steps* (breaks hidden state flow)
- Apply dropout **between layers** (in stacked RNNs)
- PyTorch: `nn.LSTM(dropout=0.3)` applies inter-layer dropout automatically

Truncated BPTT

- Full BPTT on a 1000-token sequence is too expensive
- **Truncated BPTT**: split into chunks of k steps (e.g. k=35), backprop within each chunk
- PyTorch does this by default when you call `loss.backward()` on a chunk

Common Debugging Checklist

- Loss is NaN? $\rightarrow$ gradient clipping + lower LR
- Loss plateaus? $\rightarrow$ try orthogonal init, more layers
- Overfitting? $\rightarrow$ add dropout (0.2–0.5)
- Forget gate bias = 0? $\rightarrow$ set to 1.0

LSTM: Your Brain During a Lecture

Imagine listening to a 2-hour lecture...

Your brain does **not** record every word.
Instead, it makes **smart decisions** each moment:

F Forget useless details
"The professor coughed" — not important, discard.

I Store important information
"The exam covers chapters 3–5" — write it down!

O Recall what's needed now
Someone asks "What's on the exam?" — you retrieve ch. 3–5.

An LSTM does exactly this — at every time step, it forgets, stores, and retrieves.
The "notebook" ($\boldsymbol{C}_t$) persists across the *entire* sequence.

Your Brain	LSTM
Forget boring details	Forget gate $\boldsymbol{f}_t$
Write down key facts	Input gate $\boldsymbol{i}_t$
Your long-term notes	Cell state $\boldsymbol{C}_t$
Answer a question	Output gate $\boldsymbol{o}_t$
What you say aloud	Hidden state $\boldsymbol{h}_t$

Real-life example:

You hear: *"The plane was flying normally... **suddenly**..."*

Keep: "plane flying normally"
Forget: routine details
Update: something unexpected is coming!

The LSTM Story in Plain English

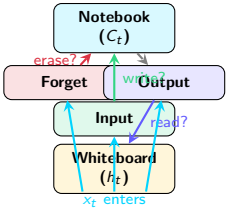
Imagine you are taking notes in a lecture...

You have a **notebook** (cell state C_t) and a **whiteboard summary** (hidden state h_t).

At each sentence the professor says, you make **3 decisions**:

1. **Forget gate:** "Is anything in my notes now irrelevant?"
   Prof changes topic from cats to dogs ⇒ erase "cat" notes.
2. **Input gate:** "What new info should I write down?"
   Prof says "the dog is friendly" ⇒ write "dog, friendly" in notebook.
3. **Output gate:** "If someone asks me right now, what do I say?"
   Not everything in the notebook is relevant to answer. Pick what matters now.

That's the entire LSTM. Three yes/no decisions at every step, applied to a notebook that persists across the whole sequence.



Intuition: Why RNNs Forget and How to Fix It

The RNN Memory Problem

An RNN's "notebook" gets **overwritten** at every step. After many steps, early information is washed away.

Analogy: Taking notes on a **single sticky note**. Every new sentence forces you to *erase and rewrite*. After 50 sentences, the first one is gone.

The LSTM Idea: Sticky Note + Filing Cabinet

- Sticky note ($\boldsymbol{h}_t$) = short-term; filing cabinet ($\boldsymbol{C}_t$) = long-term
- 3 gates: **forget** (throw away), **input** (file new), **output** (read out)

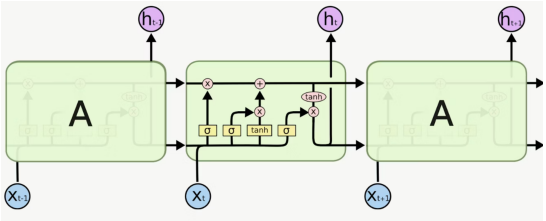
Vanilla RNN
1 sticky note
Overwrites every step
$\Rightarrow$ forgets early info

LSTM
sticky note + cabinet
Decides what to keep, file, and read out
$\Rightarrow$ remembers long-range

LSTM: Solving the Vanishing Gradient

Core Idea

Add a **cell state** C_t that flows through time with *additive* updates (no repeated multiplication).



Three gates control information flow:

1. **Forget gate** f_t – what to erase
2. **Input gate** i_t – what to write
3. **Output gate** o_t – what to expose

All gates use σ (output $\in [0, 1]$), acting as soft switches.

LSTM: The Complete Equations

Let $[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t]$ denote the concatenation of previous hidden state and current input.

Gates

$$\boldsymbol{f}_t = \sigma\big(\mathbf{W}_f[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_f\big) \quad \text{(forget)}$$

$$\boldsymbol{i}_t = \sigma\big(\mathbf{W}_i[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_i\big) \quad \text{(input)}$$

$$\boldsymbol{o}_t = \sigma\big(\mathbf{W}_o[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_o\big) \quad \text{(output)}$$

State Updates

$$\tilde{\boldsymbol{C}}_t = \tanh\big(\mathbf{W}_C[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_C\big) \quad \text{(candidate)}$$

$$\boldsymbol{C}_t = \boldsymbol{f}_t \odot \boldsymbol{C}_{t-1} + \boldsymbol{i}_t \odot \tilde{\boldsymbol{C}}_t \quad \text{(cell)}$$

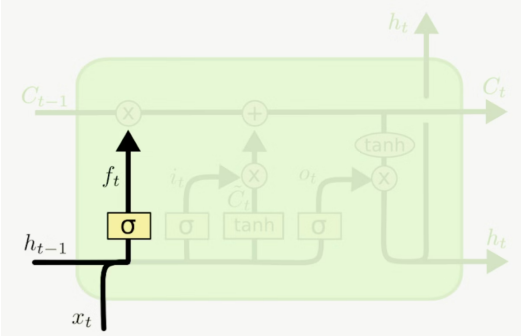
$$\boldsymbol{h}_t = \boldsymbol{o}_t \odot \tanh(\boldsymbol{C}_t) \quad \text{(hidden)}$$

$\odot$ = element-wise multiplication. Cell state update is **additive** $\Rightarrow$ gradients flow without vanishing.

Forget Gate: What to Erase

$$\boldsymbol{f}_t = \sigma\big(\mathbf{W}_f[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_f\big)$$

- Output: a vector of values in $[0, 1]$
- $f_t^{(j)} \approx 0$: **forget** dimension j of $\boldsymbol{C}_{t-1}$
- $f_t^{(j)} \approx 1$: **keep** dimension j of $\boldsymbol{C}_{t-1}$



Intuition

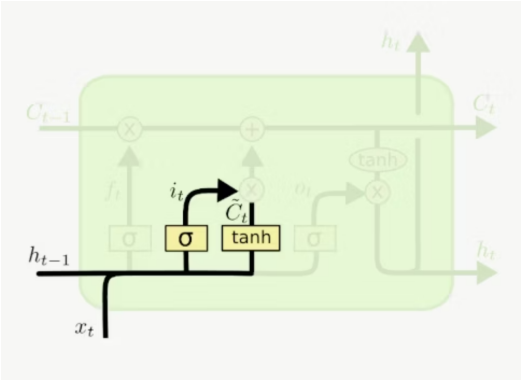
Reading "The **cat** sat on the mat. **The dog** ran away."
When we encounter "The dog", the forget gate erases the subject "cat" from memory.

Input Gate & Candidate: What to Write

$$\boldsymbol{i}_t = \sigma\big(\mathbf{W}_i[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_i\big)$$

$$\tilde{\boldsymbol{C}}_t = \tanh\big(\mathbf{W}_C[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_C\big)$$

- $\tilde{\boldsymbol{C}}_t$: **candidate** values (what *could* be stored)
- $\boldsymbol{i}_t$: **filter** deciding which candidates to accept



Intuition

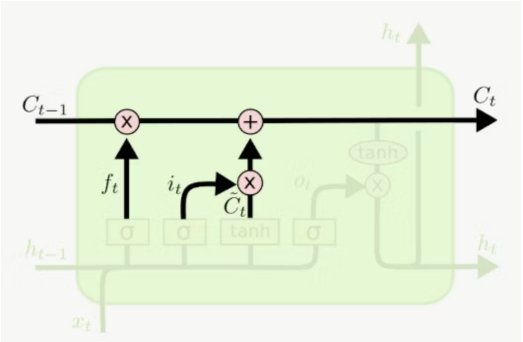
After forgetting "cat", the input gate writes "dog" as the new subject into the cell state.

Cell State Update: The "Conveyor Belt"

Cell State Update

$$C_t = \underbrace{f_t \odot C_{t-1}}_{\text{filtered old}} + \underbrace{i_t \odot \tilde{C}_t}_{\text{selected new}}$$

- Additive update $\Rightarrow$ **stable gradient flow**
- Old info is *scaled* by f_t, not multiplied by a weight matrix
- New info is *gated* by i_t before being added



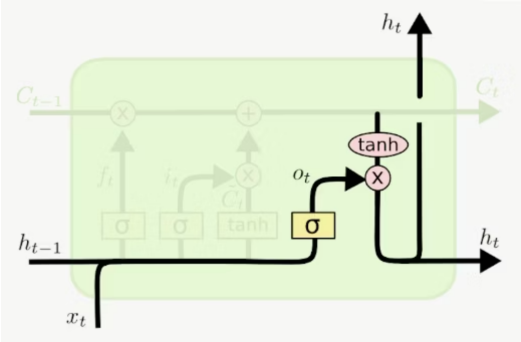
Output Gate: What to Expose

Output Gate

$$\boldsymbol{o}_t = \sigma\big(\mathbf{W}_o[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_o\big)$$

$$\boldsymbol{h}_t = \boldsymbol{o}_t \odot \tanh(\boldsymbol{C}_t)$$

- $\tanh(\boldsymbol{C}_t)$ normalises cell values to $[-1, 1]$
- $\boldsymbol{o}_t$ selects which dimensions to expose
- $\boldsymbol{h}_t$ is the **contextual embedding** at step t



Why is this better than a vanilla RNN?

- RNN: overwrites memory with a *matrix multiply* $\Rightarrow$ info decays
- LSTM: **adds** to memory with gates $\Rightarrow$ info persists
- The cell state is a **highway** that carries info across 100+ steps

One-sentence summary

At each step, the LSTM **forgets** irrelevant info, **writes** new info, **updates** the notebook, and **reads out** what's needed now.

The notebook (cell state) is the secret: it flows through time with **addition, not multiplication**.

Follow one word through the LSTM: "*The cat sat on the mat. The dog ran away.*"

LSTM Trace: Setup

Goal

Trace one LSTM step with **concrete numbers** so every operation is visible.

Dimensions: input $m=2$, hidden $d=2$. Concatenated input $[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] \in \mathbb{R}^4$.

Given (from previous step):

$$\boldsymbol{h}_{t-1} = \begin{bmatrix} 0.5 \\ -0.3 \end{bmatrix}, \quad \boldsymbol{C}_{t-1} = \begin{bmatrix} 0.8 \\ -0.2 \end{bmatrix}$$

Current input:

$$\boldsymbol{x}_t = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix}$$

Toy weights (same for all gates for simplicity):

$$\mathbf{W} = \begin{bmatrix} 0.1 & 0.2 & 0.3 & 0.4 \\ 0.5 & 0.1 & 0.2 & 0.3 \end{bmatrix}$$

Biases: $\boldsymbol{b}_f = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, $\boldsymbol{b}_i = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, $\boldsymbol{b}_C = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$,

$$\boldsymbol{b}_o = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Concatenated: $[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] = [0.5,\ -0.3,\ 1.0,\ 0.5]$

LSTM Trace: Gates (Step-by-Step)

Linear part (same **W** for all gates in this toy example):

$$\mathbf{W} \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] = \begin{bmatrix} 0.1 & 0.2 & 0.3 & 0.4 \\ 0.5 & 0.1 & 0.2 & 0.3 \end{bmatrix} \begin{bmatrix} 0.5 \\ -0.3 \\ 1.0 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 0.05-0.06+0.30+0.20 \\ 0.25-0.03+0.20+0.15 \end{bmatrix} = \begin{bmatrix} 0.49 \\ 0.57 \end{bmatrix}$$

Forget gate $\boldsymbol{f}_t$

$\sigma(0.49) = \mathbf{0.62}$
$\sigma(0.57) = \mathbf{0.64}$

Both ≈ 0.6: keep $\sim$60% of old cell state.

Input gate $\boldsymbol{i}_t$

$\sigma(0.49) = \mathbf{0.62}$
$\sigma(0.57) = \mathbf{0.64}$

Allow $\sim$60% of candidate through.

Output gate $\boldsymbol{o}_t$

$\sigma(0.49) = \mathbf{0.62}$
$\sigma(0.57) = \mathbf{0.64}$

Expose $\sim$60% of cell state.

Candidate $\tilde{\boldsymbol{C}}_t$

$\tanh(0.49) = \mathbf{0.45}, \quad \tanh(0.57) = \mathbf{0.52}$

LSTM Trace: Cell State & Hidden State

Step 1: Update cell state

$$\boldsymbol{C}_t = \boldsymbol{f}_t \odot \boldsymbol{C}_{t-1} + \boldsymbol{i}_t \odot \tilde{\boldsymbol{C}}_t$$

$$= \begin{bmatrix}0.62\\0.64\end{bmatrix} \odot \begin{bmatrix}0.8\\-0.2\end{bmatrix} + \begin{bmatrix}0.62\\0.64\end{bmatrix} \odot \begin{bmatrix}0.45\\0.52\end{bmatrix} = \begin{bmatrix}0.50\\-0.13\end{bmatrix} + \begin{bmatrix}0.28\\0.33\end{bmatrix} = \begin{bmatrix}\mathbf{0.78}\\\mathbf{0.20}\end{bmatrix}$$

Step 2: Compute hidden state

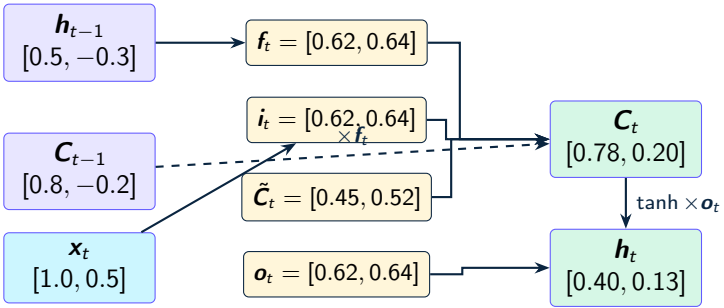
$$\boldsymbol{h}_t = \boldsymbol{o}_t \odot \tanh(\boldsymbol{C}_t) = \begin{bmatrix}0.62\\0.64\end{bmatrix} \odot \tanh \begin{bmatrix}0.78\\0.20\end{bmatrix} = \begin{bmatrix}0.62\\0.64\end{bmatrix} \odot \begin{bmatrix}0.65\\0.20\end{bmatrix} = \begin{bmatrix}\mathbf{0.40}\\\mathbf{0.13}\end{bmatrix}$$

What Just Happened

$\boldsymbol{C}_{t-1} = [0.8, -0.2] \xrightarrow{\text{forget } 38\%} [0.50, -0.13] \xrightarrow{\text{add new}} \boldsymbol{C}_t = [0.78, 0.20]$

The cell **kept** most of dim 1 (was 0.8, now 0.78) and **shifted** dim 2 from -0.2 to $+0.2$.

LSTM Trace: Summary Diagram



Key takeaway: Every number is traceable. The gates are just sigmoid/tanh applied to matrix products — nothing magic!

Why LSTM Solves Vanishing Gradients

Gradient Through the Cell State

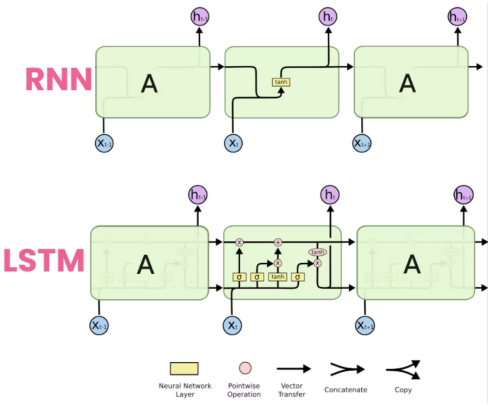
$$\frac{\partial \boldsymbol{C}_t}{\partial \boldsymbol{C}_{t-1}} = \text{diag}(\boldsymbol{f}_t)$$

- No matrix power — just element-wise scaling by $\boldsymbol{f}_t$
- When $\boldsymbol{f}_t \approx 1$: gradient passes **unchanged**
- Information can persist over **hundreds of steps**

Contrast with Vanilla RNN

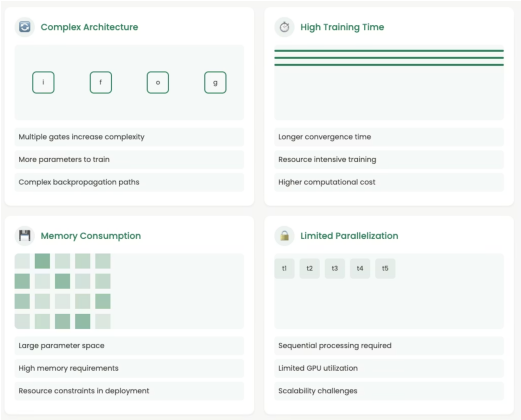
RNN: $\frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{h}_k} = \prod \boldsymbol{W}_h^\top \text{diag}(\tanh') \Rightarrow$ exp. decay

LSTM: $\frac{\partial \boldsymbol{C}_t}{\partial \boldsymbol{C}_k} = \prod \text{diag}(\boldsymbol{f}_i) \Rightarrow$ learned gates



LSTM Limitations

1. **Complex architecture** – 4 sets of weights per cell
2. **Slow training** – many parameters to optimise
3. **High memory** – stores both $\boldsymbol{C}_t$ and $\boldsymbol{h}_t$
4. **Sequential** – cannot parallelise across time steps
5. **Still limited memory** for very long sequences



Can we simplify while keeping the benefits?

$\Rightarrow$ **GRU**

GRU: Your Brain, Simplified

Same idea as LSTM, but simpler

Your brain while listening to a story:
You don't remember everything. You make **2 decisions**:

U **Update control** ("the decision knob")
"Should I stick with what I know, or change my mind?"
High $z_t \to$ *keep old memory. Low* $z_t \to$ *accept new info.*

R **Reset control** ("the forget switch")
"Should I ignore the past for this moment?"
$r_t \approx 0 \to$ *fresh start.* $r_t \approx 1 \to$ *use full history.*

GRU = smart memory that keeps important past information
and updates it when needed. **Only 2 gates** instead of LSTM's 3.

Your Brain	GRU
Keep or update?	Update gate z_t
Ignore the past?	Reset gate r_t
Your memory	Hidden state h_t

Real-life example:

You hear:
"The plane was flying normally. . .
suddenly. . . "

GRU remembers: "plane flying normally"
When "suddenly" arrives:
Update gate: blend old + new
Reset gate: how much past to use?

Result: smooth update of understanding

LSTM vs GRU

- LSTM: 3 gates + separate cell state
- GRU: **2 gates**, no separate cell state
- GRU is faster, often matches LSTM

Intuition: Can We Get the Same Benefits with Less Machinery?

The LSTM Trade-off

LSTM: **4 weight matrices**, 3 gates, 2 state vectors. Powerful but complex.

Analogy: LSTM = luxury safe with three locks. GRU = **one smart dial**.

The GRU Simplification

- **No separate cell state** — just $\boldsymbol{h}_t$
- **Reset gate**: how much past to erase
- **Update gate**: dial blending old $\boldsymbol{h}_{t-1}$ with new $\tilde{\boldsymbol{h}}_t$
- Forgetting and writing are *coupled*: what you forget = what you replace

LSTM
3 locks + drawer
4 weight sets, 2 states

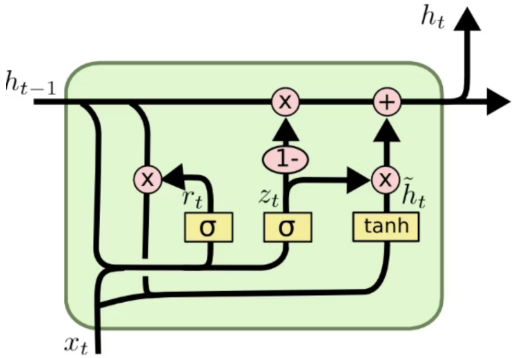
GRU
1 smart dial
3 weight sets, 1 state

GRU: A Simpler Alternative

Key Simplifications

- **No separate cell state** – merge $\boldsymbol{C}_t$ and $\boldsymbol{h}_t$
- **Two gates instead of three:**
  - **Reset gate** $\boldsymbol{r}_t$ – how much past to forget
  - **Update gate** $\boldsymbol{z}_t$ – balance old vs. new
- Fewer parameters $\Rightarrow$ faster training

	LSTM	**GRU**
Gates	3	2
State vectors	2 ($\boldsymbol{C}_t, \boldsymbol{h}_t$)	1 ($\boldsymbol{h}_t$)
Weight sets	4	3



GRU: The Complete Equations

Reset & Update Gates

$$\boldsymbol{r}_t = \sigma\big(\mathbf{W}_r[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_r\big) \qquad \text{(reset)}$$

$$\boldsymbol{z}_t = \sigma\big(\mathbf{W}_z[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_z\big) \qquad \text{(update)}$$

Hidden State

$$\tilde{\boldsymbol{h}}_t = \tanh\big(\mathbf{W}[\boldsymbol{r}_t \odot \boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}\big) \qquad \text{(candidate)}$$

$$\boldsymbol{h}_t = \boldsymbol{z}_t \odot \boldsymbol{h}_{t-1} + (1 - \boldsymbol{z}_t) \odot \tilde{\boldsymbol{h}}_t \qquad \text{(blend)}$$

Reset Gate Intuition

$\boldsymbol{r}_t \approx 0$: ignore past ("fresh start"). $\boldsymbol{r}_t \approx 1$: fully use previous state.

Update Gate Intuition

$\boldsymbol{z}_t \approx 1$: **keep** old state. $\boldsymbol{z}_t \approx 0$: **replace** with new.

GRU Trace: Concrete Numbers

Same setup: $\boldsymbol{h}_{t-1} = [0.5, -0.3]$, $\boldsymbol{x}_t = [1.0, 0.5]$, same $\mathbf{W}$, zero biases.
Linear part: $\mathbf{W} \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] = [0.49,\ 0.57]$ (as before).

Step 1: Reset gate $\boldsymbol{r}_t$

$\boldsymbol{r}_t = \sigma([0.49, 0.57]) = [0.62, 0.64]$
Both ≈ 0.6: use $\sim$60% of $\boldsymbol{h}_{t-1}$.

Step 2: Update gate $\boldsymbol{z}_t$

$\boldsymbol{z}_t = \sigma([0.49, 0.57]) = [0.62, 0.64]$
Keep $\sim$62% old, accept $\sim$38% new.

Step 3: Candidate $\tilde{\boldsymbol{h}}_t$

Reset past: $\boldsymbol{r}_t \odot \boldsymbol{h}_{t-1} = [0.31, -0.19]$
Concat with $\boldsymbol{x}_t$: $[0.31, -0.19, 1.0, 0.5]$
$\mathbf{W} \cdot [\ldots] = [0.45, 0.48]$
$\tilde{\boldsymbol{h}}_t = \tanh([0.45, 0.48]) = \mathbf{[0.42, 0.45]}$

Step 4: Final hidden state $\boldsymbol{h}_t$

$$\boldsymbol{h}_t = \boldsymbol{z}_t \odot \boldsymbol{h}_{t-1} + (1 - \boldsymbol{z}_t) \odot \tilde{\boldsymbol{h}}_t = [0.62 \times 0.5,\ 0.64 \times (-0.3)] + [0.38 \times 0.42,\ 0.36 \times 0.45] = [0.31 + 0.16,\ -0.19 + 0.16] = \mathbf{[0.47, -0.03]}$$

GRU Trace: What Changed?



Dim 1: $0.50 \to 0.47$ (barely changed, z_1=0.62 kept most)
Dim 2: $-0.30 \to -0.03$ (shifted toward 0, new info blended in)

Compare with LSTM

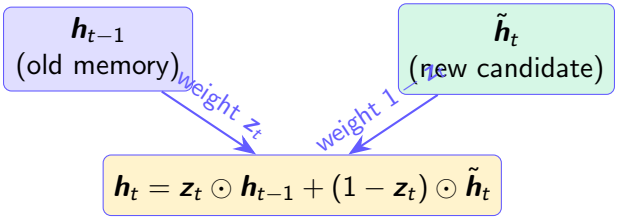
LSTM: $\boldsymbol{h}_t = [0.40, 0.13]$
GRU: $\boldsymbol{h}_t = [0.47, -0.03]$

Different values because GRU has no separate cell state — it blends directly.

Practical Insight

- GRU needed **3 computations** (vs. LSTM's 4 gates + 2 states)
- Same σ/tanh building blocks
- Fewer parameters $\Rightarrow$ faster, but less control

GRU: The Blending Mechanism



Key Insight: Coupled Forget/Input

Unlike LSTM where f_t and i_t are independent, GRU **couples** them:

"Forget old" $= z_t$, "Accept new" $= 1 - z_t$ $\Rightarrow$ always sum to 1.

RNN vs LSTM vs GRU: At a Glance

Property	Vanilla RNN	LSTM	GRU
Hidden state equation	$\tanh(\mathbf{W}\boldsymbol{x} + \mathbf{U}\boldsymbol{h} + \boldsymbol{b})$	6 equations	4 equations
Gates	0	3 (f, i, o)	2 (r, z)
Separate cell state	No	Yes ($\boldsymbol{C}_t$)	No
Long-range memory	Poor	Good	Good
Parameters	Fewest	Most	Medium
Training speed	Fast	Slow	Fast
Best for	Short sequences	Long sequences	Small data / speed

Rule of Thumb

LSTM: default choice for long sequences. **GRU**: when data is limited or speed matters. In practice, performance is often similar — **experiment!**

Common Confusions — Clearing Them Up

Confusion	Answer
"Are weights shared across time steps?"	**Yes, always.** The same $\mathbf{W}_x, \mathbf{W}_h, \boldsymbol{b}$ are reused at every t. That is what makes it *recurrent*.
"Does BiRNN double the sequence length?"	**No.** It doubles the *hidden dimension* ($d \to 2d$). Sequence length stays the same.
"Can I use LSTM for real-time streaming?"	**Yes**, but only the *forward* direction. BiLSTM needs the full sequence upfront.
"LSTM has more gates, so it is always better?"	**Not always.** GRU often matches LSTM on small/medium data and trains faster. **Experiment.**
"Why is my loss NaN after a few epochs?"	Exploding gradients. Add `clip_grad_norm_(1.0)` and lower your learning rate.

How Many Parameters? A Concrete Comparison

Setup

Input dimension m, hidden dimension d. Each gate/layer has a weight matrix for input ($d \times m$), one for hidden ($d \times d$), and a bias (d).

	Vanilla RNN	**LSTM**	**GRU**
Gates / transforms	1	4 (f, i, o, C)	3 (r, z, h)
Params per gate	$d(m+d) + d$	$d(m+d) + d$	$d(m+d) + d$
Total parameters	$d(m+d+1)$	$\mathbf{4}\, d(m+d+1)$	$\mathbf{3}\, d(m+d+1)$
Example: m=300 (GloVe), d=256			
Params (single layer)	**142 k**	**570 k**	**427 k**

Key Insight

LSTM has **4**× the parameters of a vanilla RNN. GRU has **3**×. More parameters = more capacity, but slower training and higher memory.

Bidirectional

A BiLSTM **doubles** the count (two separate LSTMs). A 2-layer BiLSTM: $\approx$ 2.3M params for m=300, d=256.

PyTorch in Practice: 10 Lines to Build RNN/LSTM/GRU

Defining the Model

```
import torch.nn as nn

# Pick one --- same API!
rnn = nn.RNN(input_size=300,
  hidden_size=256, batch_first=True)

lstm = nn.LSTM(input_size=300,
  hidden_size=256, batch_first=True,
  num_layers=2, bidirectional=True)

gru = nn.GRU(input_size=300,
  hidden_size=256, batch_first=True)
```

Running a Forward Pass

```
# x:  (batch, seq_len, 300)
out, h_n = rnn(x)
# out:  all hidden states
#  (batch, seq_len, 256)
# h_n:  last hidden state
#  (1, batch, 256)

# For LSTM, also returns cell:
out, (h_n, c_n) = lstm(x)
# h_n:  (2*2, batch, 256)
#  2 layers * 2 directions
```

Key: `nn.RNN`, `nn.LSTM`, `nn.GRU` share the same interface.
Switch between them by changing **one line**. Add `bidirectional=True` and `num_layers=N` for deep BiLSTMs.

ELMo: The First Contextual Embedding (Peters et al., 2018)

Architecture

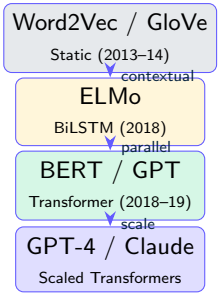
Char CNN $\rightarrow$ 2-layer BiLSTM (trained as language model) $\rightarrow$ weighted layer average.

$$\text{ELMo}_t = \gamma \sum_{\ell=0}^{L} s_\ell \, \boldsymbol{h}_t^{(\ell)}$$

s_ℓ = learned layer weights. Layer 1 $\approx$ syntax, Layer 2 $\approx$ semantics.

Impact & Limitations

- \+ **+3–25%** on 6 benchmarks by adding ELMo to existing models
- \+ Proved "**pre-train then fine-tune**" paradigm
- – Feature-based (frozen, not fine-tuned)
- – Sequential BiLSTM: slow

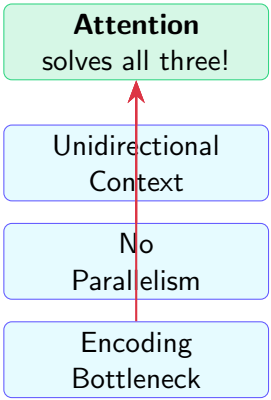


Remaining Limitations (All Recurrent Models)

1. **Encoding bottleneck** – entire sequence compressed into a fixed-size $\boldsymbol{h}_T$
2. **No parallelism** – must process tokens one by one ($O(T)$ sequential steps)
3. **Unidirectional context** – $\boldsymbol{h}_t$ only sees past, not future

What We Need

- Attend to **any position** in the sequence
- Process all tokens **in parallel**
- Capture **bidirectional** context



The Missing Piece: Attention for RNNs (Bahdanau et al., 2014)

Fixing the Bottleneck

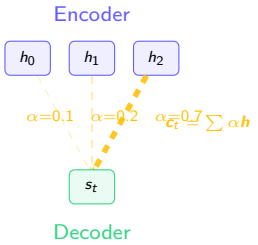
Instead of compressing the entire input into one vector $\boldsymbol{c}$, let the decoder **look back** at all encoder hidden states at each step.

How attention works:

1. Compute a **score** between decoder state $\boldsymbol{s}_t$ and each encoder state $\boldsymbol{h}_j$
2. Normalise scores to get **attention weights** $\alpha_{t,j}$ (softmax)
3. Weighted sum = **context vector** for this step

$$\alpha_{t,j} = \frac{\exp(\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_j))}{\sum_k \exp(\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_k))}, \qquad \boldsymbol{c}_t = \sum_j \alpha_{t,j}\, \boldsymbol{h}_j$$

Now $\boldsymbol{c}_t$ is **different at each decoding step** — the decoder "pays attention" to the relevant input words.



Intuition

Translating "student": pay most attention to "étudiant" (α=0.7), less to "Je" (α=0.1).

Next Lecture: Attention Is All You Need

The Transformer (Vaswani et al., 2017)

- Replaces recurrence with **self-attention**
- All positions attend to all others in **parallel**
- Foundation for BERT, GPT, T5, and modern LLMs

Key results from the paper:

- 28.4 BLEU on EN-DE translation
- 41.8 BLEU on EN-FR translation
- Trained in a **fraction** of the time vs. RNN models



>100k citations and counting!

Summary

1. **Static embeddings** give one vector per word — insufficient for polysemy
2. **RNNs** accumulate context via hidden states; can be **stacked** and made **bidirectional**
3. Vanishing gradients limit vanilla RNNs; **gradient clipping** fixes explosions only
4. **LSTMs** add a cell state with gated *additive* updates $\Rightarrow$ long-range memory
5. **GRUs** simplify to 2 gates, 75% of LSTM params, comparable performance
6. **Seq2Seq** encoder-decoder enables variable-length I/O but has a **bottleneck**
7. **Attention** (Bahdanau 2014) lets the decoder look at all encoder states
8. **ELMo** (2018): first contextual embedding via 2-layer BiLSTM
9. All recurrent models are sequential $\Rightarrow$ **Transformers** next!

Key Equations to Remember

RNN: $\boldsymbol{h}_t = \tanh(\mathbf{W}_x \boldsymbol{x}_t + \mathbf{W}_h \boldsymbol{h}_{t-1} + \boldsymbol{b})$ LSTM: $\boldsymbol{C}_t = \boldsymbol{f}_t \odot \boldsymbol{C}_{t-1} + \boldsymbol{i}_t \odot \tilde{\boldsymbol{C}}_t$ GRU: $\boldsymbol{h}_t = \boldsymbol{z}_t \odot \boldsymbol{h}_{t-1} + (1-\boldsymbol{z}_t) \odot \tilde{\boldsymbol{h}}_t$

Attention: $\boldsymbol{c}_t = \sum_j \alpha_{t,j} \boldsymbol{h}_j$

Key References

Foundational Architectures:

1. Elman (1990). *Finding structure in time*. Cognitive Science. — Simple RNN
2. Hochreiter & Schmidhuber (1997). *Long Short-Term Memory*. Neural Computation. — LSTM
3. Gers et al. (2000). *Learning to Forget*. Neural Computation. — Forget gate
4. Cho et al. (2014). *Learning Phrase Representations using RNN Encoder–Decoder*. EMNLP. — GRU
5. Sutskever et al. (2014). *Sequence to Sequence Learning with Neural Networks*. NeurIPS. — Seq2Seq
6. Schuster & Paliwal (1997). *Bidirectional RNNs*. IEEE Trans. Signal Proc. — BiRNN

Training:

7. Pascanu et al. (2013). *On the Difficulty of Training RNNs*. ICML. — Gradients & clipping

Attention & Contextual Embeddings:

8. Bahdanau et al. (2014). *Neural MT by Jointly Learning to Align and Translate*. ICLR. — Attention
9. Peters et al. (2018). *Deep Contextualized Word Representations*. NAACL. — ELMo
10. Vaswani et al. (2017). *Attention Is All You Need*. NeurIPS. — Transformer
11. Devlin et al. (2019). *BERT*. NAACL. — Bidirectional Transformer
12. Radford et al. (2018). *Improving Language Understanding by Generative Pre-Training*. — GPT

Static Embeddings:

13. Mikolov et al. (2013). *Efficient Estimation of Word Representations*. ICLR. — Word2Vec
14. Pennington et al. (2014). *GloVe*. EMNLP. — Global Vectors
15. Bojanowski et al. (2017). *Enriching Word Vectors with Subword Information*. TACL. — FastText

Recommended Reading & Resources

Textbooks:

- Jurafsky & Martin. *Speech and Language Processing*, Ch. 9 (RNNs & LSTMs). Free online: web.stanford.edu/~jurafsky/slp3/
- Goodfellow, Bengio & Courville. *Deep Learning*, Ch. 10 (Sequence Modelling). Free online: deeplearningbook.org
- Goldberg, Y. (2017). *Neural Network Methods for NLP*. Morgan & Claypool.

Tutorials & Blog Posts:

- Olah, C. (2015). *Understanding LSTM Networks*. colah.github.io — The classic visual guide.
- Karpathy, A. (2015). *The Unreasonable Effectiveness of Recurrent Neural Networks*. — Intuitive intro with code.
- Jay Alammar. *The Illustrated Transformer* & *Illustrated ELMo*. — Visual explanations.

Code:

- PyTorch RNN Tutorial: pytorch.org/tutorials
- HuggingFace `transformers`: huggingface.co/docs

Questions?

Thank you!

Prof. Anis Koubaa
akoubaa@alfaisal.edu

Alfaisal University